Conceptos Avanzados en el Modelado de Bases de Datos

Generalización, Especialización, Agregación y Atributos con Histórico

Jaqueline Diaz Velasco
Tarea de Bases de Datos

Septiembre 2026

Resumen

El Modelo Entidad-Relación Extendido (EER) introduce construcciones semánticas para representar estructuras complejas en el diseño de bases de datos relacionales. Este documento profundiza en cuatro conceptos fundamentales: la abstracción bottom-up (Generalización), el refinamiento top-down (Especialización), las relaciones de abstraimiento estructural (Agregación) y las estrategias para el manejo de atributos dependientes del tiempo (Atributos con Histórico).

1 Generalización y Especialización

En el diseño relacional avanzado, la generalización y la especialización permiten establecer jerarquías de herencia tipo "es-un" (*is-a*) entre entidades supertipo y subtipo.

1.1 1. Generalización (Enfoque Ascendente / Bottom-Up)

La **generalización** es el proceso sintético de identificar características comunes (atributos y relaciones) entre múltiples conjuntos de entidades para sintetizarlas en una entidad de nivel superior denominada **Superclase** o **Supertipo**.

- **Propósito:** Minimizar la redundancia definiendo los atributos comunes una sola vez en el supertipo.
- **Ejemplo:** Si en el sistema existen las entidades `Automóvil(Placa, Color, NumPuertas)` y `Motocicleta(Placa, Color, Cilindrada)`, se generalizan en la superclase `Vehículo(Placa, Color)`.

1.2 2. Especialización (Enfoque Descendente / Top-Down)

La **especialización** es el proceso analítico inverso a la generalización. Consiste en dividir una entidad de alto nivel (Supertipo) en varias entidades de menor nivel (**Subtipos**), basándose en diferencias en sus atributos o relaciones.

- **Propósito:** Modelar atributos específicos que solo aplican a un grupo reducido de entidades.
- **Ejemplo:** De la entidad `Empleado(ID, Nombre, Salario)` se especializan las subclases `Empleado_Por_Hora(TarifaHora)` y `Empleado_Asalariado(SueldoMensual)`.

1.3 Restricciones en Jerarquías de Herencia

1. **Disyunción vs. Solapamiento:**
   - **Disjunta (Disjoint):** Una instancia del supertipo puede pertenecer a un solo subtipo (ej. un empleado es de hora O asalariado, no ambos).
   - **Solapada (Overlapping):** Una instancia puede ser de varios subtipos simultáneamente (ej. una persona puede ser Estudiante y Empleado).
2. **Totalidad vs. Parcialidad:**
   - **Total:** Toda entidad del supertipo debe pertenecer obligatoriamente a algún subtipo.
   - **Parcial:** Pueden existir entidades en el supertipo que no pertenezcan a ningún subtipo.

2 Relaciones de Agregación

El modelo relacional estándar no permite abstraer una relación entre entidades como si fuera una entidad en sí misma. La **Agregación** resuelve esta limitación sintáctica.

2.1 Definición

La agregación es una abstracción a través de la cual una **relación entre dos o más entidades se trata como una entidad compuesta de mayor nivel**, permitiéndole participar en otras relaciones adicionales.

2.2 Caso de Uso y Ejemplo

Supongamos que la entidad `Profesor` imparte la entidad `Materia`. La unión de ambas forma una relación de impartición (`Imparte`). Si queremos registrar los `Exámenes` aplicados a esa combinación específica de *Profesor-Materia*, no podemos asociar `Examen` únicamente a `Materia` o solo a `Profesor`.

- Se agrega la relación Profesor $\leftrightarrow$ Materia en la entidad compuesta `Asignación`.
- Posteriormente se crea la relación: Asignación $\xrightarrow{\text{Aplica}}$ Examen.

3 Atributos con Histórico (Datos Temporales)

En las bases de datos transaccionales convencionales, los atributos almacenan únicamente el **estado actual**. Sin embargo, en muchos dominios de negocio se requiere conservar el registro de cambios en el tiempo.

3.1 Estrategias de Implementación

Para almacenar atributos con histórico existen tres métodos principales:

1. **Tablas de Auditoría/Histórico:** Creación de una tabla secundaria que almacena la traza de cambios mediante disparadores (*triggers*).

2. **Modelado Temporal Nativo (Versionado con Fechas):** Incluir columnas de rango de validez (`fecha_inicio`, `fecha_fin`) en la tabla principal.
3. **Tipos de Datos de Rango en PostgreSQL:** Uso de tipos como `TSTZRANGE` o `DATERANGE`.